

Protocol Audit Report

Version 1.0

Cyfrin.io

August 10, 2026

Protocol Audit Report

CJ

August 3, 2026

Prepared by: [CJ] Lead Securuty Reasercher: - CJ

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - The fundings described in this document correspond the following commit hash:**
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - High
    - [H-1] Storing the password on-chain makes it visable to anyone
    - [H-2] `PasswordStored::setPassword` has no access controls, meaning a non-owner could change the password
  - Medium
  - Low

    * [I-1] The `PasswordStore::getPassword` natspec indicates a prameter that doesn't exist, causing the natspect to be incorrect.
  - Informational
  - Gas

Protocol Summary

Protocol does X, Y, Z

Disclaimer

The CJ team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
	High	H	H/M	M
Likelihood	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The fundings described in this document correspond the following commit hash:**

```
2e8f81e263b3a9d18fab4fb5c46805ffc10a9990
```

Scope

```
./src/
#-- PasswordStore.sol
```

Roles

- Owner: The user who can set the password and read the password.
- Outsider: No one else should be able to set or read password.

Executive Summary

Add some notes about how the audit went,types of things you found, etc. We spent X hours with Z auditors using Y tools. etc

Issues found

Severity	Number of issues found
High	2
Medium	0
Low	0
Info	1
Total	3

Findings

High

[H-1] Storing the password on-chain makes it visable to anyone

Description: All data stored on-chain are visible to anyone,and can be read directly from the blockchain.The `PasswordStored ::s_password` is intended to be a private variable and only

accessed through the `PasswordStored::getPassword` function, which is intended to be only called by owner of the contract.

We show a one such method of reading any data off chain below.

Impact: Anyone can read the password,severely breaking the functionality of the protocol.

Proof of Concept: The below test case shows how anyone can read the password directly from the blockchain .

1. Create a locally running chain

```
make anvil
```

2. Deploy the contract on chain

```
make deploy
```

3. Run the storage tool

We use 1 because that's the storage slot of `s_password` in the contract.

```
cast storage <Address_HERE> 1 --rpc-url http://127.0.0.1:8545
```

You will get the output like this:

`0x6d7950617373776f726400000000000000000000000000000000000000000014`

You can then parse that hex to a string with:

```
cast parse-bytes32-string 0
    x6d7950617373776f726400000000000000000000000000000000000000000014
```

And get output looks like:

```
myPassword
```

Recommended Mitigation:

Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain to decrypt the password on-chain. This would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view funciton as you wouldn't the user accidentally send a transaction with the password that decrypts your passwords.

[H-2] `PasswordStored::setPassword` has no access controls, meaning a non-owner could change the password

Description: The `PasswordStored::setPassword` function is set to be an `external` funciton, however, the natspec of the function and overall purpose of the smart contract is`This function allows only the owner to set a new password.`

```
function setPassword(string memory newPassword) external {
@>      // @audit There is no access control
        s_password = newPassword;
        emit SetNetPassword();
    }
```

Impact: Anyone can set the stored password, severely breaking the contract's intended functionality.

Proof of Concept: Add the following to the `PasswordStored.t.sol` test file.

Code

```
function test_non_owner_can_set_password(address randomAddress) public
    {
        vm.assume(randomAddress != owner);
        vm.prank(randomAddress);
        string memory expectedPasssword = "myNewPassword";
        passwordStore.setPassword(expectedPasssword);

        vm.prank(owner);
        string memory actualPassword = passwordStore.getPassword();
        assertEq(actualPassword, expectedPasssword);
    }
```

Recommended Mitigation: Add an access control conditional to the`setPassword` function.

```
if(msg.sender != owner) {
    revert PasswordStored__NotOwner();
}
```

Medium

Low

[I-1] The `PasswordStore::getPassword` natspec indicates a prameter that doesn't exist, causing the natspect to be incorrect.

Informational

Gas